网络编程

黑马程序员
www.itheima.com
传智教育旗下
高端IT教育品牌

目录

Contents

- 网络编程三要素

  IP地址、端口、协议、3次握手、4次挥手

- Socket套接字与TCP开发流程

  服务器端程序流程、客户端程序流程

- TCP编程

  服务器端程序、客户端程序编写

目录

Contents

- 网络编程三要素

  IP地址、端口、协议

- Socket套接字与TCP开发流程
- TCP编程

学习目标

Learning Objectives

1. 知道IP地址的作用
2. 知道端口和端口号的作用
3. 能够说出TCP的特点

基本概念

网络

将具有独立功能的多台计算机通过通信线路和通信设备连接起来，在网络管理软件及网络通信协议下，**实现资源共享和信息传递的虚拟平台**。

能够**编写基于网络通信的软件或程序**，通常来说就是网络编程。

基本概念

要使用编程语言实现多台计算机的网络通信，需要具备网络编程**三个要素**：

(1) **IP地址**：这是网络环境下每一台计算机的唯一标识，通过IP地址来找到指定的计算机；

(2) **端口**：用于标识进程的逻辑地址，通过端口来找到指定的进程；

(3) **协议**：定义通信规则，符合协议则可以通信，否则无法正常通信。

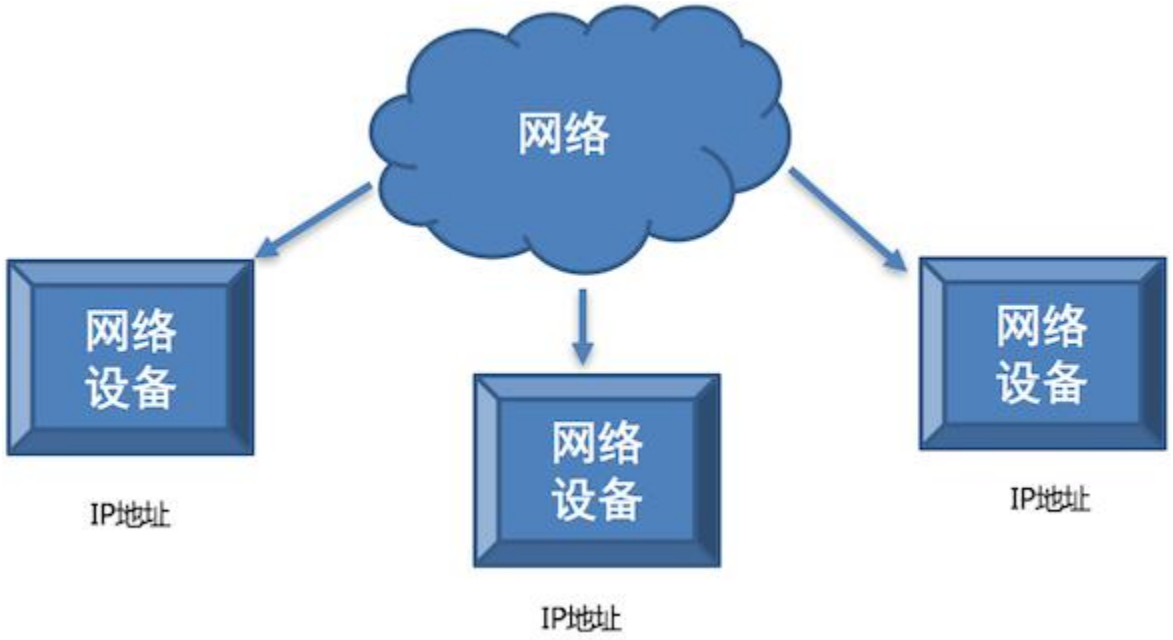
例如，假如我要和小明面对面说上话。

- 首先我要到【**小明的住处**】找到小明(相当于通过IP地址找到指定计算机)；
- 然后我要和小明说话，小明使【**用耳朵听**】我说(相当于用端口接收)；
- 最后，我们对话不能随便使用任意语言，此时说的话【**彼此都能听懂**】，如讲普通话(这就是协议的作用)。

IP地址

IP地址的概念

IP地址就是**标识网络中设备的一个地址**，好比现实生活中的家庭地址。



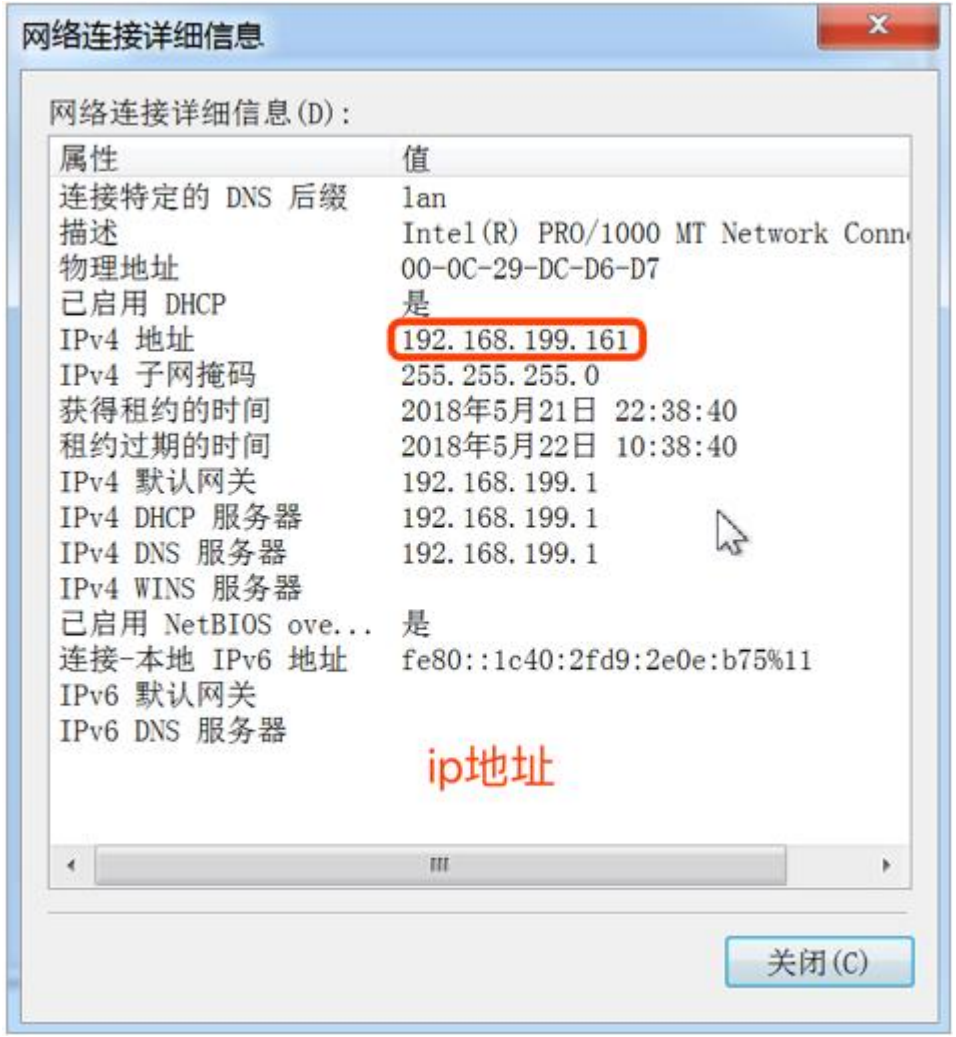
IP地址

IP地址的表现形式

IP地址分为两类： IPv4和IPv6

IPv4：是目前大家使用的IP地址；

IPv6：作为了解，IPv6是未来使用的IP地址。



IP地址

IP地址的作用

通过**IP地址找到网络中唯一一台设备**，也就是说通过IP地址能够找到网络中某台设备，然后可以跟这个设备进行数据通信。

IP地址

查看IP地址与检查网络

Linux 和 mac OS 使用 ifconfig 这个命令

Windows 使用 ipconfig 这个命令

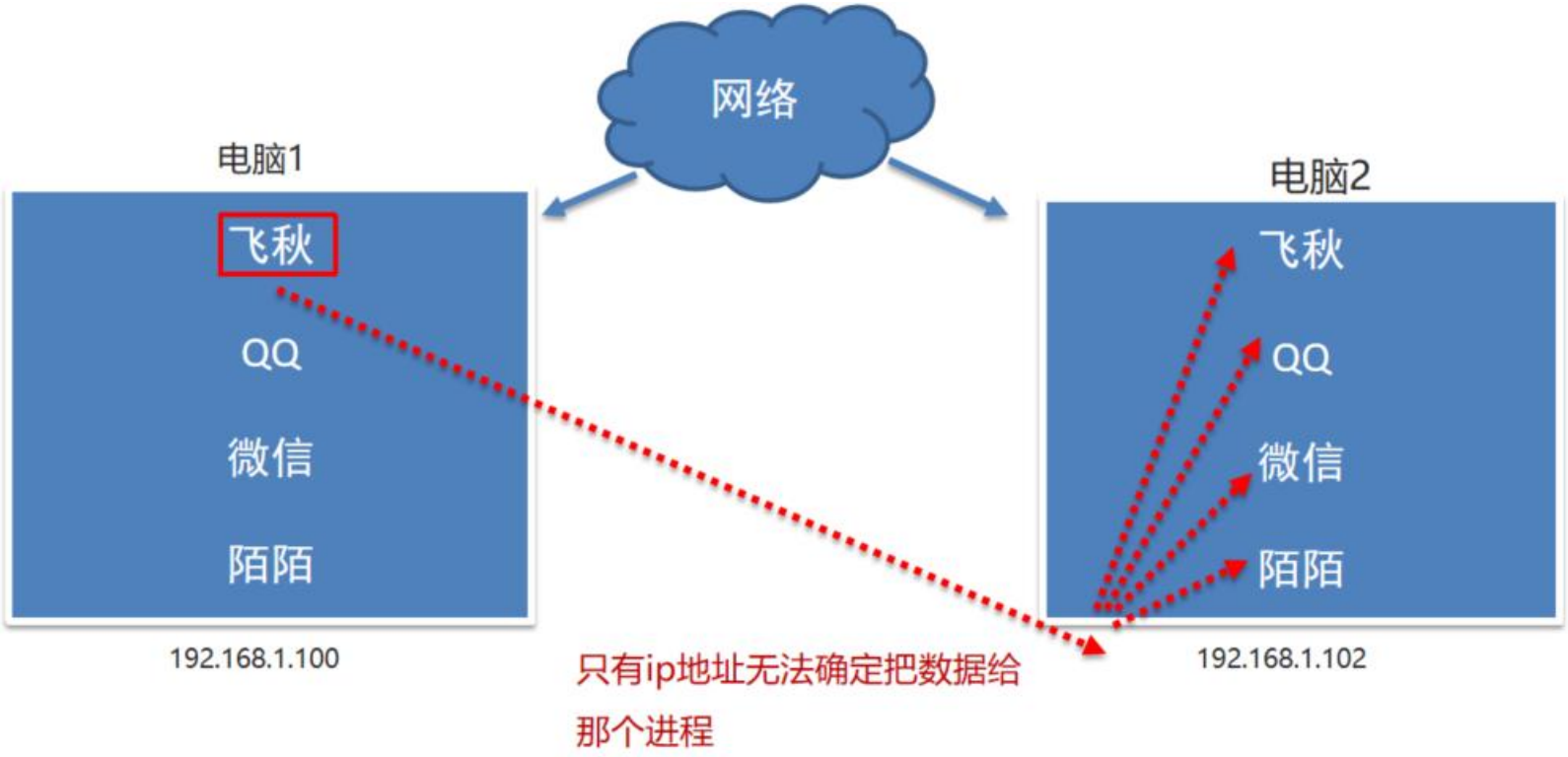
检查网络是否正常使用 ping 命令

- ping www.baidu.com 检查是否能上公网
- ping 当前局域网的ip地址 检查是否在同一个局域网内
- ping 127.0.0.1 检查本地网卡是否正常

端口和端口号

问题思考

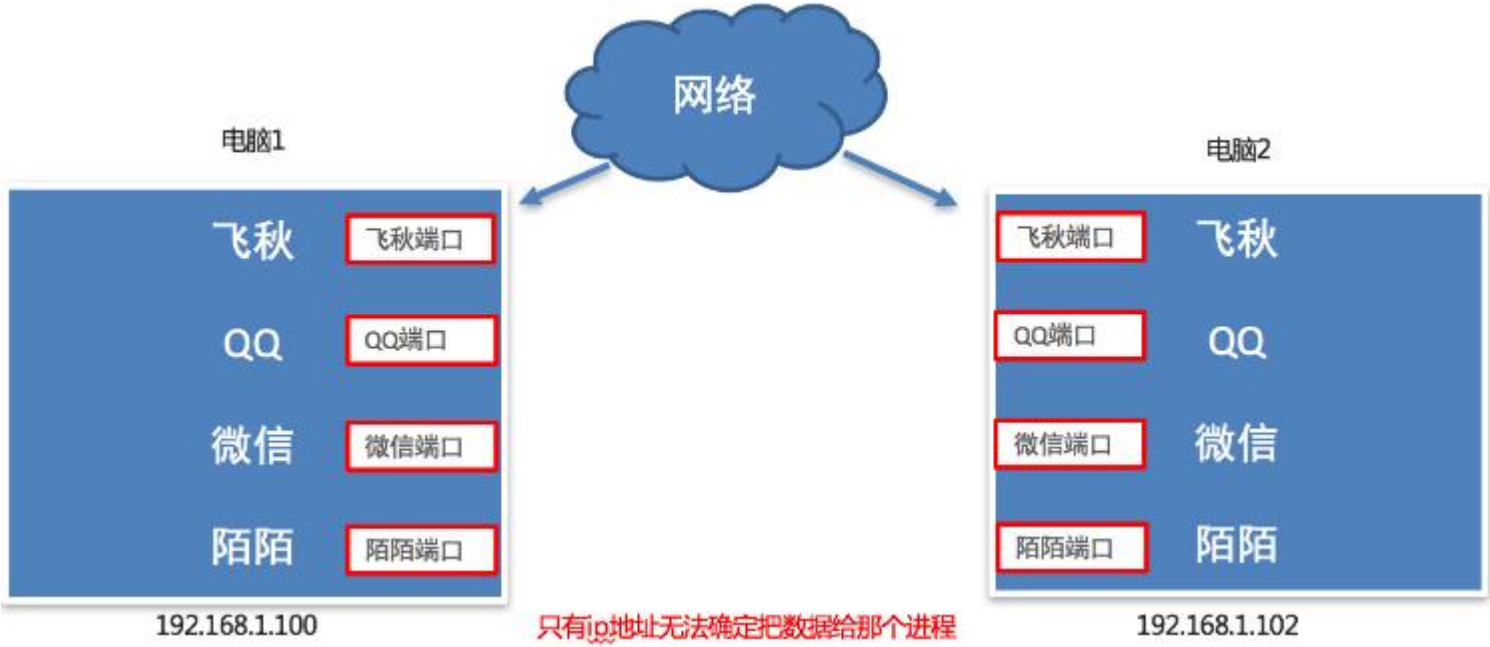
在一台电脑上使用飞秋给另外一台电脑上的飞秋发送数据并且另外的这台电脑还运行着多个软件，它是如何区分这多个软件把数据给飞秋的呢?



端口和端口号

什么是端口

其实，**每运行一个程序都会有一个端口，想要给对应的程序发送数据，找到对应的端口即可**。



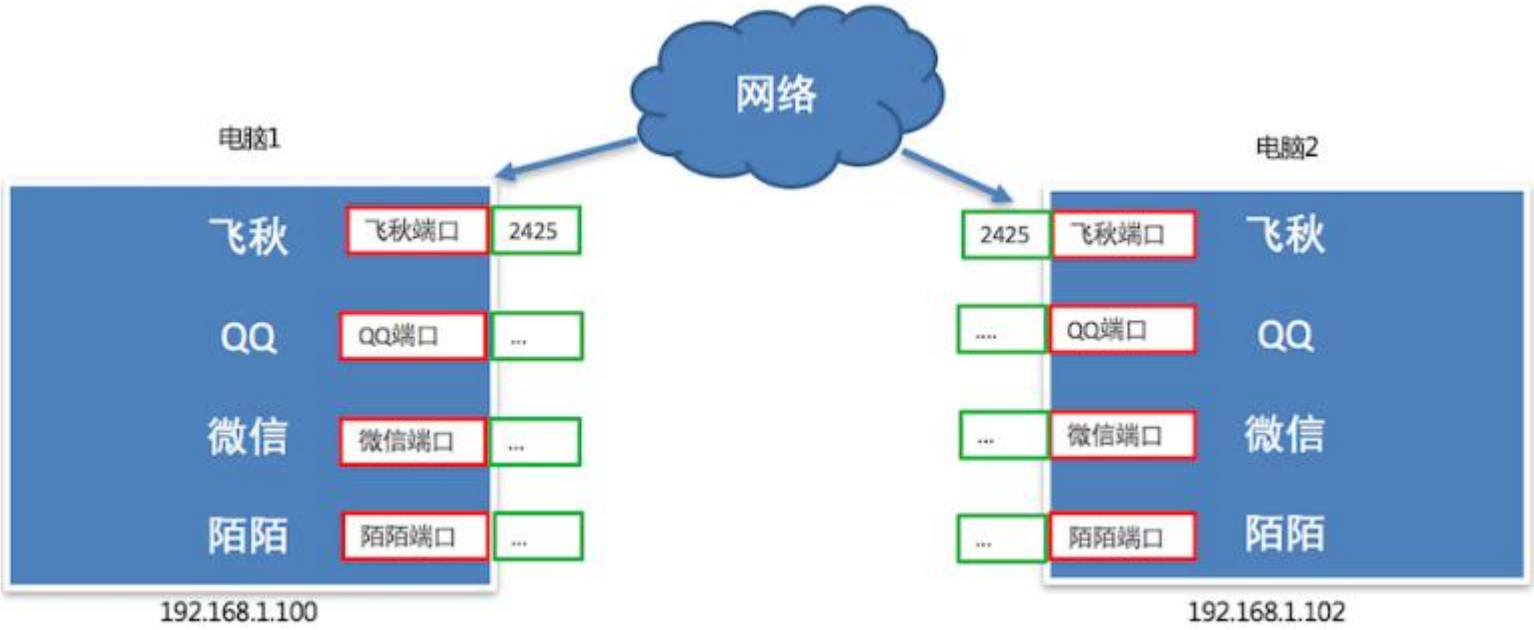
端口是传输数据的通道，好比教室的门，**是数据传输必经之路**

注：给已经运行起来的程序分配端口号；没有运行的程序

端口和端口号

什么是端口号

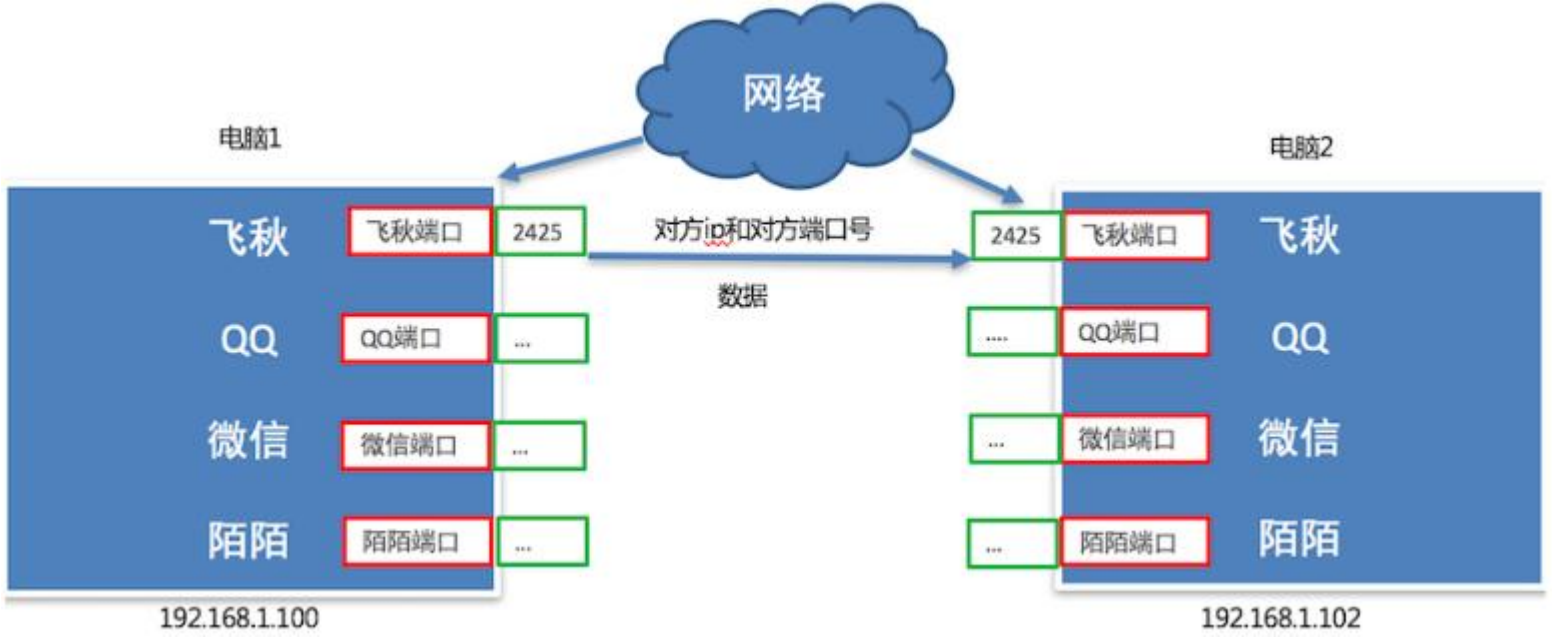
其实，**每一个端口都会有一个对应的端口号，想要找到端口通过端口号即可。**



端口和端口号

端口和端口号的关系

端口号可以标识电脑中唯一的一个端口。



端口和端口号

端口号的分类

知名端口号：是指众所周知的端口号，范围从0到1023。

动态端口号：一般程序员开发应用程序使用端口号称为动态端口号，范围是从1024到65535。

TCP协议

通信流程

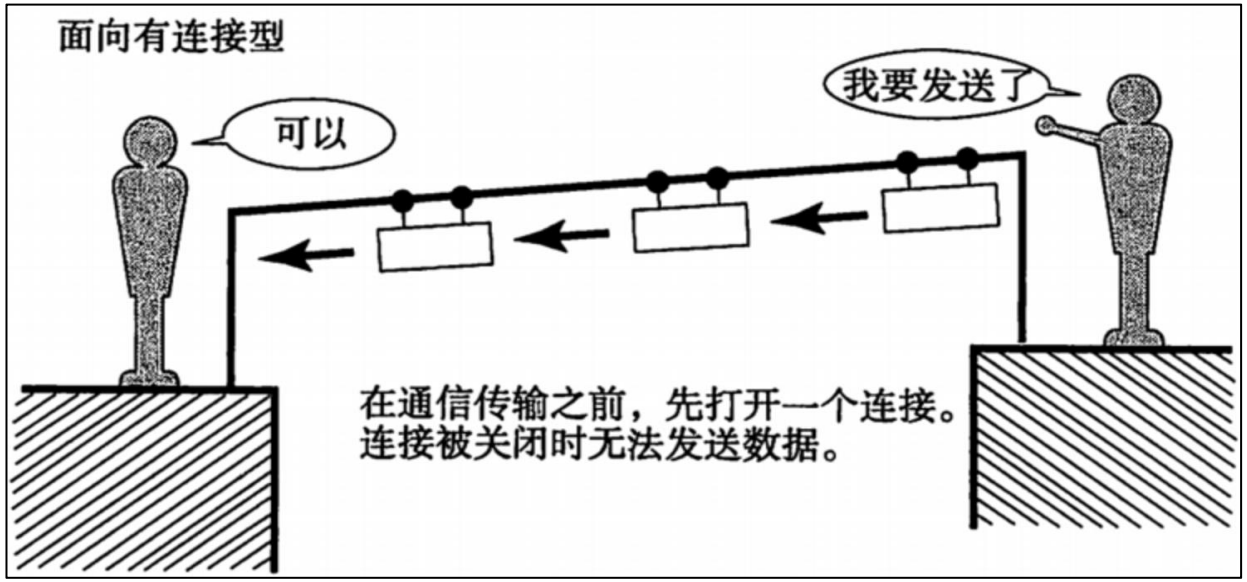
之前我们已学习了 IP地址和端口号，通过 IP地址能够找到对应的设备，然后再通过端口号找到对应的端口，再通过端口把数据传输给应用程序。**要注意，数据不能随便发送，在发送之前还需要规则，以保证程序之间按照指定的规则来进行数据的通信**，而这个规则就是TCP协议。

TCP的概念

TCP的英文全拼(Transmission Control Protocol)简称传输控制协议，**它是一种面向连接的、可靠的、基于字节流的传输层通信协议。**

TCP协议

TCP面向连接



TCP 通信步骤:

创建连接

传输数据

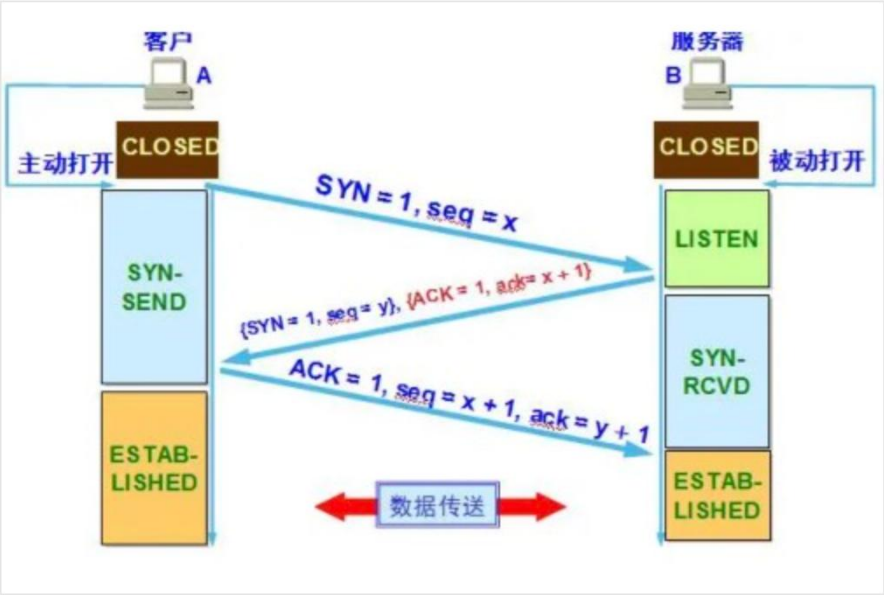
关闭连接

说明:

TCP通信模型相当于生活中的’打电话‘，在通信开始之前，一定要先建立好连接，才能发送数据，通信结束要关闭连接

TCP协议创建连接：3次握手

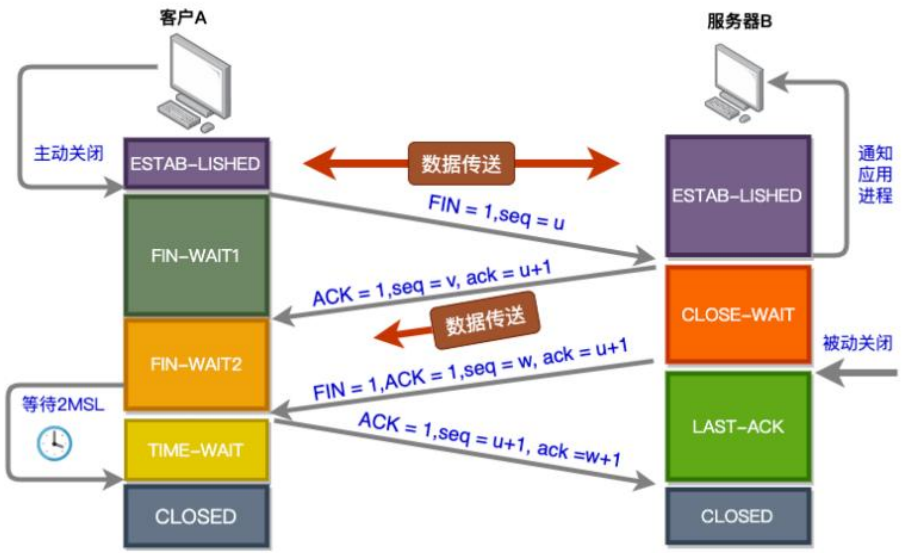
三次握手（Three-Way Handshake）就是指建立一个TCP连接时，需要客户端和服务端总共发送3个包以确认连接的建立。



- 第一次握手：客户端向服务端发送请求，**等待服务端确认**。
- 第二次握手：服务端收到请求后**知道客户端请求建立连接，回复给客户端以确认连接请求**。
- 第三次握手：**客户端收到确认后**，再次发送请求确认服务端，服务端收到正确请求后，如果正确则连接建立成功，完成三次握手，随后客户端与服务端之间可以开始传输数据了。

TCP协议断开连接：4次挥手

四次挥手说TCP断开链接的时候需要经过4次确认。TCP连接是双向，A连接B、B连接A都要断开



- 第一次挥手： 当主机A（可以是客户端也可以是服务端）完成数据传输后，提出停止TCP 连接的请求
- 第二次挥手： 主机B收到请求后对其作出响应，确认这一方向上的TCP连接将关闭
- 第三次挥手： 主机B 端再提出反方向的连接关闭请求
- 第四次挥手： 主机A对主机B 的请求进行确认，双方向的关闭结束

TCP协议

TCP的特点

面向连接

- 通信双方必须先建立好连接才能进行数据传输，数据传输完成后，需要断开连接，以释放系统资源。

可靠传输

- 都建立了连接，传输数据可靠
- 必须要先建立连接，相对效率较低
- 大多数服务器程序都是使用TCP协议开发的，比如文件下载，网页浏览等

总结

1. 网络编程三要素是哪些？

   IP地址、端口号、协议

2. IP地址的作用？

   网络中电脑设备的唯一标识

   查看：ipconfig，查看网络是否连接：ping

3. 端口号的作用？

   ①公有端口：0-1023；②动态端口：1024-65535

4. TCP协议？

   必须先建立连接，再传输数据；可靠，类似于“打电话”

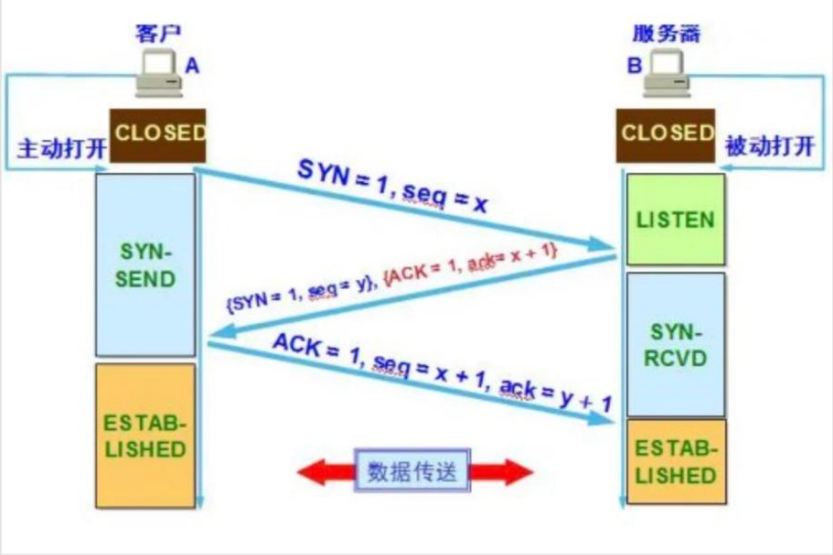
总结

5 三次握手大致流程？

第1次握手： A发送请求（随机数x）

第2次握手： B回消息（x+1， y），A确认x

第3次握手：A回复消息（y+1），B确认y



总结

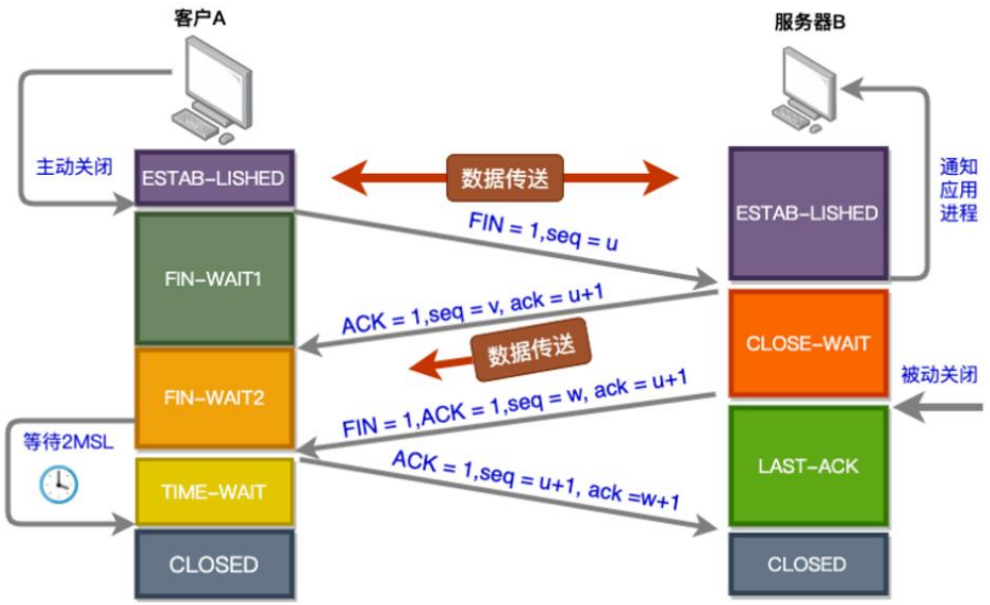
6 四次挥手大致流程？

第1次挥手： A发送请求（ 随机数u）

第2次挥手： B回消息（U+1）A确认U；关闭A到B的连接

第3次挥手： B发送消息（U+1，w）

第4次挥手：A回复消息（U+1，w+1）B确认w；关闭B到A的连接



目录

Contents

- 网络编程三要素
- Socket套接字与TCP开发流程

  概念、客户端、服务器端socket开发流程、str与二进制转换
- TCP编程

学习目标

Learning Objectives

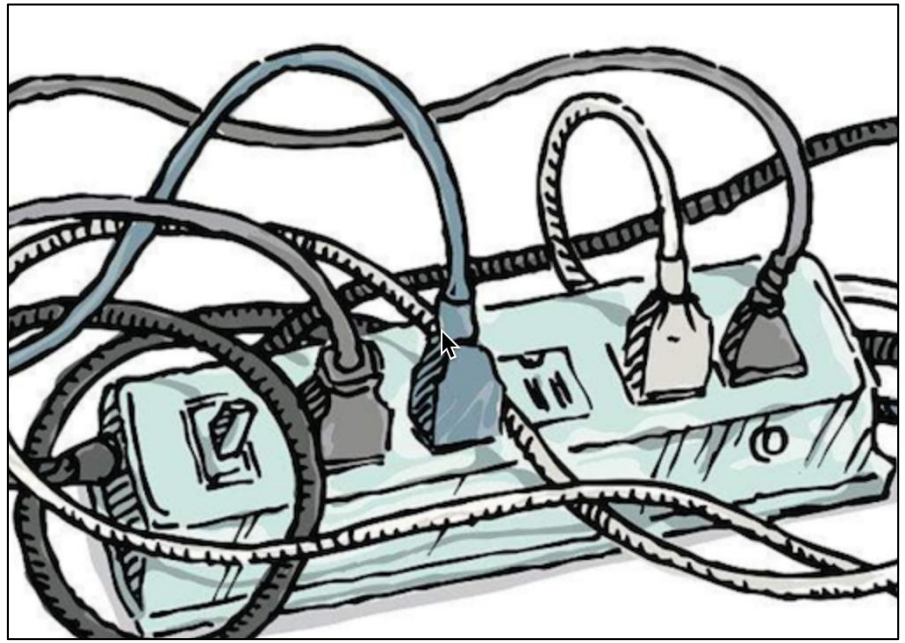
1. 了解socket套接字的作用
2. 知道TCP程序的开发流程

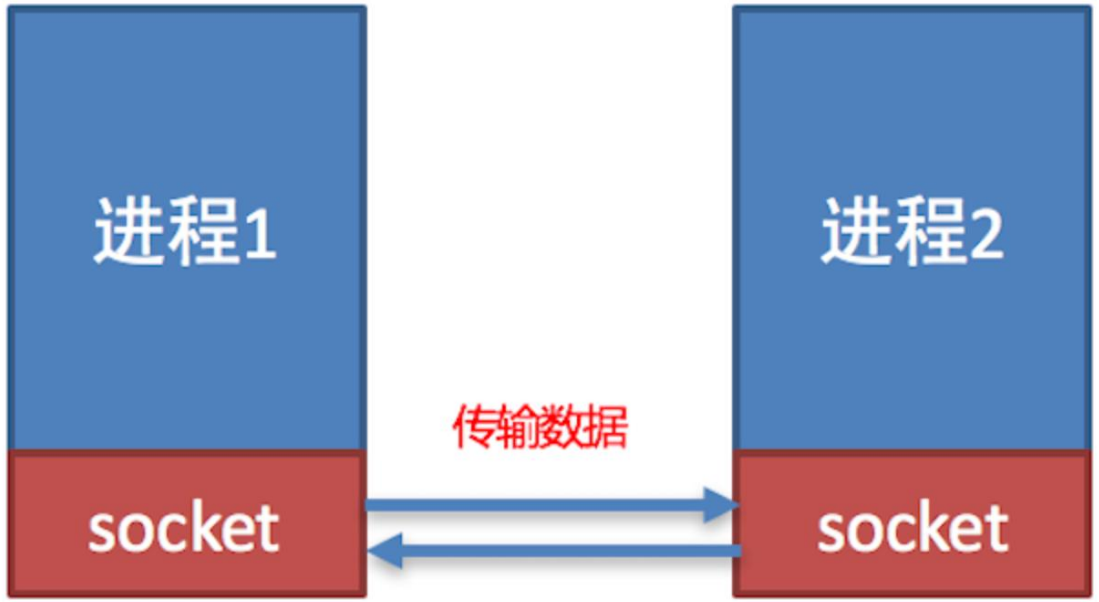
socket套接字

socket的概念

知道网络编程的三要素，数据是如何完成传输的呢？此时，就可以使用 **socket**来完成。

socket(简称 套接字) 是**进程之间通信一个工具**，好比现实生活中的**插座**，所有的家用电器要想工作都是基于插座进行，而**进程之间想要进行网络通信需要基于这个 socket**。





socket套接字

socket使用场景

负责**进程之间的网络数据传输**，好比数据的搬运工。

不夸张的说，**只要跟网络相关的应用程序或者软件都使用到了socket**。

socket套接字

使用socket

socket(套接字)能实现不同主机之间的进程间通信。Python中有专门的socket类:

```
# 导入socket模块
import socket
```

要使用socket，则通常要使用到socket模块下的socket类创建socket对象：

函数名	含义
socket(AddressFamily, Type)	用于创建一个socket对象。其中，参数AddressFamily可以选择AF_INET用于Internet进程间通信，实际工作中常用它；参数Type表示套接字类型，可以是SOCK_STREAM流式套接字，主要用于TCP协议。

socket套接字

使用socket

案例分析

例如，来创建一个TCP协议下的socket对象。

```
# 导入模块
import socket

# 创建socket对象
tcp_socket = socket.socket(socket.AF_INET,socket.SOCK_STREAM)

# 输出内容
print(tcp_socket)
```

TCP程序开发流程

TCP开发流程介绍

TCP网络应用程序开发分为:

- TCP客户端程序开发（ps: 浏览器）
- TCP服务端程序开发

说明:

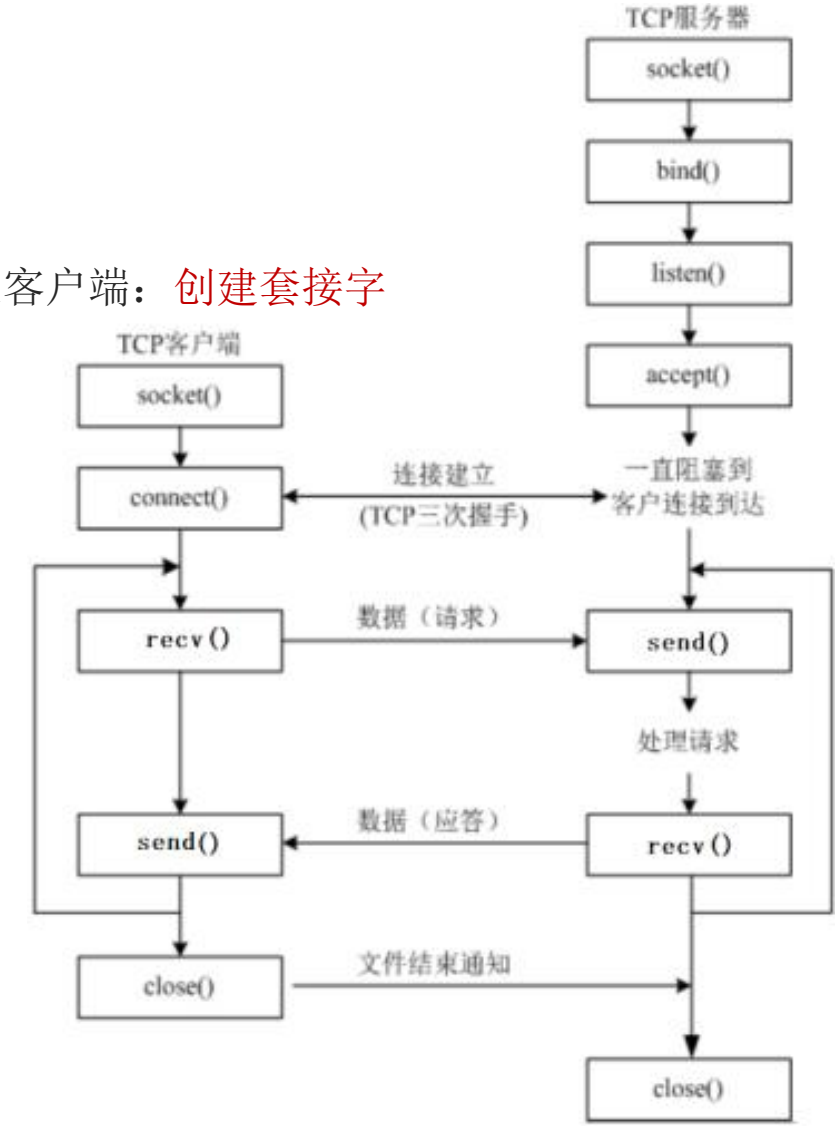
客户端程序是指运行在**用户设备上的程序**

服务端程序是指运行在**服务器设备上的程序**，专门为客户端提供数据服务。

TCP程序开发流程

TCP开发流程介绍

在客户端：创建套接字



在服务器端创建套接字：也叫被动套接字，只负责接受新来的客户端连接

在服务器端：accept()会返回一个新的套接字，用来和客户端收发消息

服务器端：关闭和客户端1、客户端2、客户端n的连接套接字。（不主动关闭）

服务器端：关闭被动套接字（不主动关闭）

TCP程序开发流程

TCP服务器端

TCP服务器端操作**步骤流程说明**：

1. 创建服务端套接字对象
2. 绑定端口号
3. 设置监听
4. 等待接受客户端的连接请求
5. 发送数据
6. 接收数据
7. 关闭套接字

TCP程序开发流程

TCP服务器端

当有了socket对象后，常用于服务器端的函数如下：

函数名	含义
bind(address)	绑定地址(host,port)到socket套接字，在AF_INET下，address以元组(host,port)的形式表示地址。其中，参数host表示字符串类型的主机名或IP地址；port表示int类型的端口号。
listen(backlog)	开始TCP监听。参数backlog指定在拒绝连接之前，操作系统可以挂起的最大连接数量。该值至少为1，大部分应用程序设为5就可以了，最大值可设置为128。
accept()	被动接受TCP客户端连接，等待连接的到来，返回一个connection对象，以元组形式显示，且元组的第一个参数为已建立连接的socket对象，第二个参数为地址address。

能同时应用于服务器端和客户端的函数如下：

函数名	含义
recv(bufsize)	用于接收TCP数据，数据以bytes二进制形式返回，bufsize指定要接收的最大数据量，可设置为1024。
send(data)	发送TCP数据，将data中的数据发送到连接的套接字，data为bytes二进制数据。
close()	关闭套接字。
setsockopt(level,optname,value)	可设置端口号复用，让程序退出端口号并立即释放。level可指定为SOL_SOCKET, optname可指定为SO_REUSEADDR, value可指定为True。

TCP程序开发流程

TCP客户端

TCP客户端操作步骤流程：

1. 创建客户端套接字对象
2. 客户端连接服务器端
3. 接收数据
4. 发送数据
5. 关闭套接字

当有了socket对象后，常用于客户端的函数如下：

函数名	含义
connect(address)	主动初始化TCP服务器连接，一般地，参数address的格式为元组(host,port)，若连接出错，则返回socket.error错误。

TCP程序开发流程

字符串str与二进制bytes类型转换

在网络中，**数据是以`二进制数据类型bytes`的形式进行传递的**，所以在我们向网络传输数据的时候需要把数据转化成`二进制`，从网络中接受到的数据默认也是`二进制`类型的数据，想要正常使用这些数据也需要把这些数据从`二进制`类型数据转化为`字符串str`型。

当字符串str数据转换为二进制bytes类型时，可使用如下函数：

encode(encoding)	用于把字符串编码转换为二进制数据，其中，encoding表示编码格式，常设置为utf-8。

当二进制bytes数据转换为字符串str类型时，可使用如下函数：

decode(encoding)	用于把二进制数据解码转换为字符串，其中，encoding表示编码格式，常设置为utf-8。

TCP程序开发流程

字符串str与二进制bytes类型转换

案例分析

例如，把字符串`黑马程序员`转换为二进制数据（此过程：编码）

把b`itheima`（b打头表示二进制数据）转换为字符串（此过程：解码）。

```
strs = "黑马程序员"
# 编码为二进制
bs = strs.encode(encoding="utf-8")
print(bs)
# 解码为字符串
result = bs.decode(encoding="utf-8")
print(result)
```

```
bs = b"itheima"
# print(bs)
# print(type(bs))
# 解码为字符串
result = bs.decode(encoding="utf-8")
print(result)
```

总结

1. 网络通信使用socket套接字；
2. TCP网络应用程序开发分为客户端和服务端程序开发；
3. 网络传输数据类型是二进制bytes

目录

Contents

- 网络编程三要素
- Socket套接字与TCP开发流程
- **TCP编程**

服务器端程序、客户端程序编写

学习目标

Learning Objectives

1. 能完成客户端与服务器端发送或接收消息
2. 了解开发TCP网络应用程序的注意点

服务器端

案例分析

例如，创建一个socket对象服务器端，并向客户端发送消息内容Welcome to study socket!

```
import socket

if __name__ == '__main__':
    # 1.创建服务端套接字对象
    tcp_server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # 2. 绑定IP地址和端口号
    # tcp_server_socket.bind(("172.16.62.136", 8888))
    # 如果bind中的参数第一个ip地址元素设置为"",默认为本机ip地址
    tcp_server_socket.bind(("", 8888))
    # 3.设置监听128:代表服务端等待排队连接的最大数量
    tcp_server_socket.listen(128)
```

```
    # 4.等待接受客户端的连接请求accept阻塞等待返回一个用以和客户端通socket,客户端的地址
    conn_socket, ip_port = tcp_server_socket.accept()
    print("客户端地址:", ip_port)
    # 5.接收数据
    recv_data = conn_socket.recv(1024)
    print("接收到的数据:", recv_data.decode())
    # 6.发送数据
    conn_socket.send("客户端你的数据我收到了".encode())
    # 7.关闭套接字
    conn_socket.close()
    tcp_server_socket.close()
```

客户端

案例分析

例如，创建一个socket对象客户端，并用来接收服务器端发送过来的内容。

```
import socket

if __name__ == '__main__':
    # 1.创建客户端套接字对象
    tcp_client_socket = socket.socket(socket.AF_INET,
socket.SOCK_STREAM)
    # 2.和服务端套接字建立连接
    tcp_client_socket.connect(("172.16.62.136",8080))
    # 3.发送数据
    tcp_client_socket.send("nihaome".encode(encoding="utf-8"))
    # 4.接收数据recv阻塞等待数据的到来
    recv_data = tcp_client_socket.recv(1024)
    print(recv_data.decode())
    # 5.关闭客户端套接字
    tcp_client_socket.close()
```

说明：

当客户端和服务端建立连接后，**服务端程序退出后端口号不会立即释放，需要等待大概1-2分钟。**

解决办法有两种：

- 更换服务端端口号
- 设置端口号复用(推荐大家使用)，也就是说让服务端程序退出后端口号立即释放。

设置端口号复用的代码如下：

```
# 参数1: 表示当前套接字 # 参数2: 设置端口号复用选项 # 参数3: 设置端口号复用选项对应的值
tcp_server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, True)
```

TCP网络程序的注意点

1. **TCP服务端程序必须绑定端口号**，否则客户端找不到这个TCP服务端程序，为了更稳定，建议把IP也绑定
2. accept()前的套接字是被动套接字，**只负责接收新的客户端的连接请求，不能收发消息**

3. 当 TCP客户端程序和 TCP服务端程序连接成功后， TCP服务器端程序会产生一个**新的套接字**，用于收发客户端消息
4. 若关闭 accept()返回的被动连接套接字，则表示和这个客户端已经通信完毕

5. 对于服务器端socket，**关闭时需慎重**
6. 当客户端的套接字调用 close 后，服务器端的 recv 会解阻塞，返回的数据长度为0，**用于判断客户端是否已经下线**

总结

1. 服务器端代码实现；

2. 客户端代码实现。

黑马程序员
www.itheima.com
传智教育旗下高端IT教育品牌